

A Formal Software Engineering Methodology for Semantically Aligned Digital Twins

Name, Last Name
University
{name.last_name}@uni.com
City, Country

Abstract—Safety-critical cyber-physical systems increasingly rely on digital twins (DTs) for monitoring, prediction, and decision-making alongside their physical counterparts (PTs). For DT-based reasoning to be correct, the DT must be semantically aligned with the PT: the twins must agree on the meaning assigned to observed data, and their observable behaviors must correspond. These properties are implicitly assumed throughout the DT literature, yet neither is formally defined nor guaranteed by any existing approach. We present an MBSE-grounded software engineering framework for establishing and preserving semantic alignment between PT and DT, requiring only a shared ontology as their interface. We formally define semantic alignment between PT and DT models and show that it yields formal guarantees in the form of consistency in data interpretation and equivalence of observable behaviors under the shared ontology, which are checkable at design time, inherited at the implementation level and preserved across ontology evolution.

Index Terms—digital twins, semantic alignment, ontologies

I. INTRODUCTION

The development of safety-critical cyber-physical systems (CPS) follows an iterative process in which requirements, models, and implementations co-evolve across design cycles, driven by changing regulations, design improvements, and emerging optimization opportunities. Model-based Systems Engineering (MBSE) [1] is the established discipline for managing this complexity in industries such as automotive and aerospace [2], [3], [4], [5], [6], providing a rigorous framework that captures system structure, behavior, and requirements, and through which correctness can be formally established and preserved across iterations.

The concept of Digital Twins (DTs) [7] has emerged as a powerful software engineering paradigm to support decision-making across the CPS lifecycle. A DT pairs a virtual representation of a deployed system, referred to as its Physical Twin (PT), with a bidirectional data connection to it [8], enabling run-time monitoring, prediction, and decision support throughout the system’s operational life [9], [10], [11], [12]. Their use has been extended beyond monitoring and predictive analytics to support iterative design refinement, virtual prototyping, and post-deployment adaptation [13], [14], [15].

In practice, PT and DT are two different engineering artifacts, often developed by different teams and owned by different stakeholders, with different internal vocabularies and optimized for different purposes (one for physical realization,

the other for monitoring and decision support). As a result, their natural development trajectories diverge and both evolve independently across the system lifetime. This separation of concerns allows for scalable independent evolution as well as federation in modern architectures. In Gaia-X [16], for example, independently developed DTs from different organizations reason jointly over common physical infrastructure, interoperating through shared ontologies without exposing internal implementation details.

For DT-based reasoning to be correct, it must be semantically aligned with its PT throughout the system’s lifetime. That is, two properties must hold: the twins must agree on the meaning assigned to observed data, and their observable behaviors must correspond. A DT that misinterprets a sensor reading or exhibits behavior inconsistent with the PT is unsound and in safety-critical domains, any decision or safety guarantee derived from it is invalid. These properties are implicitly assumed throughout the DT literature [7], [9], [10], yet no existing approach establishes them jointly and over all possible behaviors [17].

To bridge this gap, we present a formal software engineering framework for establishing and preserving semantic alignment between the DT and PT across design iterations and system evolution. Grounded in MBSE and requiring only a shared ontology as the semantic interface between twins, we formally define semantic alignment as consistency in the interpretation of observed data and equivalence of observable behaviors, yielding formal guarantees of data and behavioral consistency. We show that these guarantees can be established at design-time and are inherited by deployed implementations, guaranteeing semantic alignment for the lifetime of the system without the engineering cost of any runtime mechanism.

This paper is structured as follows. Section II motivates our approach via a running example. Section III presents the framework. Section IV provides the necessary formal background used in Section V and VI to formally define the domain knowledge and semantic alignment. Section VIII evaluates the framework and Section IX discusses related work. Finally, Section X concludes the paper.

II. MOTIVATING EXAMPLE

As part of a large nation-wide smart farming initiative, several companies collaborate to automate crop monitoring

and pesticide application across large agricultural fields using fleets of autonomous drones. These companies operate independently, with proprietary concerns and development processes. To support federation and interoperability, the system operates under Gaia-X, where each company interoperates through shared ontologies, exposing only the relevant aspects of their systems without revealing internal details.

In these settings, the company operating the drone fleet develops a DT to monitor and optimize the drones' behavior, reducing energy consumption, that must also remain compliant with environmental regulations, monitored independently via a separate DT operated by a compliance auditor.

This architecture surfaces a fundamental problem. Each DT must maintain a correct interpretation of the PT's behavior, while allowing independent development and interoperability. For the energy management DT, this means agreeing with the PT on the meaning of battery and flight status, and being able to mimic all of its behaviors, soundly predicting the drone's future actions, or risking optimizations the drone cannot safely execute, with consequences for any downstream stakeholder, such as the compliance auditor.

III. MULTI-VIEW FRAMEWORK

We ground our framework for semantic alignment in the concept of views [1], [3], [6], a foundational abstraction mechanism in MBSE [18], [19] through which a system is formally described from multiple stakeholder perspectives. A view captures a specific concern, such as timing, functional behavior, or safety constraints, as a formal model, enabling concern-specific reasoning and mitigating the complexity of understanding the system as a whole. Views are the authoritative design-time artifacts in MBSE against which properties are verified and from which implementations are generated.

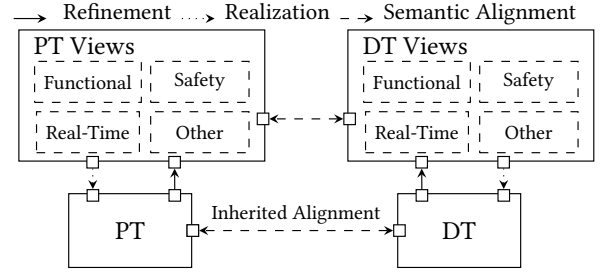
As shown in Figure 1, we distinguish two kinds of views. PT-views are standard MBSE views of the physical system, owned by PT engineers and stakeholders, capturing its concerns fully and authoritatively. DT-views, on the other hand, are developed independently, potentially by different teams and organizations, and address the subset of PT concerns relevant to monitoring, prediction, and decision support. Each DT-view must be shown to be aligned with its corresponding PT-view, ensuring specification-level coherence on the relevant subset of behaviors the DT is expected to reason about.

To maintain consistency between abstraction levels, any implementation must refine its views, exhibiting only behaviors they permit [1].

IV. BACKGROUND

The framework of Section III operates in safety-critical domains, where the system's behavior must be formally verified against specifications. Here, correctness is inherently time-dependent: the right events must occur, and within the required deadlines. Timed Transition Systems (TTS) and Timed Automata (TA) [20], [21] provide the semantic and syntactic foundations for modeling such timed behavior. Correctness properties are expressed in Timed Computation Tree Logic

Fig. 1: Multi-view Framework



(TCTL) [20] over a set of atomic propositions (APs), which is expressive enough to capture the requirements of safety-critical systems and whose model checking against TAs is decidable and efficiently supported by tools such as UPPAAL [22]. This combination is the established standard for safety-critical verification, with applications in automotive [23], railway [24], and avionics [25], as well as DT applications [26], [27], [28], making it the natural formal substrate for our framework.

We report the definition of TTS below; TA and TCTL are standard and we refer to [21] for their definitions.

Definition 1. A TTS is a tuple $(S, s_0, \Sigma_E, \Sigma_D, \rightarrow, L)$, where:

- S is a set of states, where s_0 is the initial state, and
- Σ_E is a set of *event labels*,
- Σ_D is a set of *delay labels*, with $\delta \in \Sigma_D$ representing the elapse of δ time units.
- $\rightarrow \subseteq S \times (\Sigma_E \cup \Sigma_D) \times S$ is the transition relation.
- $L : S \rightarrow 2^{AP}$ is a labeling function that assigns to each state a set of APs that hold in that state.

To reason about the behavioral relationship between two TTS, we use weak timed simulation (WTS) and weak timed bisimulation (WTB) [29], [30]. Given two TTS $\mathcal{T}_1$ and $\mathcal{T}_2$, $\mathcal{T}_2$ simulates $\mathcal{T}_1$, written $\mathcal{T}_1 \preceq_X \mathcal{T}_2$, when every observable behavior of $\mathcal{T}_1$ over alphabet X and every timing event can be matched by $\mathcal{T}_2$. Bisimulation $\mathcal{T}_1 \stackrel{X}{\sim} \mathcal{T}_2$ strengthens this to a two-way relationship: the two systems are behaviorally indistinguishable over X . Events outside X are treated as internal and abstracted away as τ -transitions. We write $s \xrightarrow{\tau^*} s'$ if s' is reachable from s by any sequence of τ -transitions and delay transitions, and $s \xrightarrow{a} s'$ for $a \in \Sigma_E \setminus \{\tau\}$ if there exist s_1, s_2 such that $s \xrightarrow{\tau^*} s_1 \xrightarrow{a} s_2 \xrightarrow{\tau^*} s'$.

Both lift to TAs via their semantics: $\mathcal{A}_1 \preceq_X \mathcal{A}_2$ iff $\llbracket \mathcal{A}_1 \rrbracket \preceq_X \llbracket \mathcal{A}_2 \rrbracket$, and analogously for $\sim$. When X is the total alphabet of both systems we drop the subscript. Both relations are decidable [29], [30] and monotone in X : $\mathcal{A}_1 \stackrel{Y}{\sim} \mathcal{A}_2$ implies $\mathcal{A}_1 \stackrel{X}{\sim} \mathcal{A}_2$ for $X \subseteq Y$, similarly for $\preceq$. As standard results [21]: bisimulation preserves all TCTL properties over X , and simulation transfers safety properties from $\mathcal{T}_2$ to $\mathcal{T}_1$.

V. FORMALIZATION OF DOMAIN KNOWLEDGE

Engineers design systems against an understanding of the application domain, i.e., the entities, relationships, and constraints that govern its behavior. Ontologies are the standard

tool for representing this knowledge in DT architectures [31], [32], [33]. We generalize this notion to a first-order theory.

Definition 2. An ontology is a tuple $K = (S, F, R, \Delta)$ where:

- S is a finite set of value domains, representing the types of entities in the domain;
- F is a finite set of function symbols with declared value domains signatures, representing measurable quantities and domain-specific computations;
- R is a finite set of relation symbols with declared value domains signatures, representing domain predicates;
- Δ is a finite set of axioms, expressing what the engineer knows to be true and invariant about the domain. $\Delta \subseteq \mathcal{F}(S, F, R)$, where $\mathcal{F}(S, F, R)$ is the set of all first-order sentences over the signature.

The signature (S, F, R) defines the vocabulary of the domain. A model of K is an instantiation of the vocabulary satisfying Δ . We write $\mathcal{L}(K)$ for the set of all facts expressible by the vocabulary entailed by Δ . Consider the example in Section II. A possible ontology $K = (S, F, R, \Delta)$ for the drone domain, capturing the relevant domain knowledge about field size, battery capacity, and energy consumption, is as follows:

- $S = \{\text{Hec}, \mathbb{R}, \mathbb{N}\}$, capturing units of land, real-valued energy, and natural-valued hectare indices.
- $F = \{\text{cons} : \text{Hec} \rightarrow \mathbb{R}, \text{num_hec} : \mathbb{N}, \text{tot_con} : \mathbb{R}, \text{tot_bat} : \mathbb{R}\}$, where $\text{cons}(h)$ is the energy needed to cover Hec h , and the remaining symbols are constants.
- $R = \{\text{done} : \text{Hec}\}$, where $\text{done}(h)$ holds when hectare h has been sprayed.
- $\Delta = \{\text{num_hec} = 10, \text{tot_bat} = 500 \text{ Wh}, \forall h : \text{Hec}. \text{cons}(h) \geq 0, \text{tot_con} = \sum_h^{\text{done}(h)} \text{cons}(h), \text{tot_con} \leq \text{tot_bat}\}$

This representation does not require engineers to produce new knowledge, as it is already present in existing engineering artifacts, such as system documentation, domain standards, and engineering models, which can be used to automatically construct the ontology [34], [35], [36], [37], [38], [39]. Our framework merely promotes this knowledge to a first-class reasoning artifact that can participate in refinement analysis. We represent it as a first-order theory rather than knowledge graphs or description logics, the dominant choice in DT applications [31], [32], [33], because the knowledge encoded in safety-critical domains is inherently quantitative: certification bounds, energy budgets, timing constraints, and subsystem interface requirements cannot be expressed as taxonomies or subclass relations. Deriving that a payload constraint implies a flight stability bound requires arithmetic reasoning over physical quantities, beyond the expressivity of both formalisms. Many decidable fragments of first-order logic exist that are sufficiently expressive for such domains (e.g., quantifier-free linear arithmetic [40], full linear real arithmetic [41], Presburger arithmetic [42], [43], and the effectively propositional fragment, which subsumes knowledge graphs [44]) and in practice solvers handle combinations of those efficiently [45].

We assume throughout that K is expressed in such a fragment. Moreover, we focus on ontologies capturing facts invariant across all operating conditions and system states, keeping the domain layer independent from the behavioral layer.

When engineers write TCTL properties, they abstract their understanding of the domain into APs that reflect e.g. actuator commands, threshold crossings, or compliance events. This abstraction is well motivated by the need for tractable model-checking and assumes a connection between these labels and the domain knowledge. Any verification claim over any AP presupposes a decision about what that AP means in terms of the physical system, as evidenced by tools such as FRET [46] and STIMULUS [47], which already require engineers to provide explicit AP bindings to system requirements. We formalize this connection as the interpretation function I , defined as follows.

Definition 3. Given a TTS $\mathcal{T}$, an interpretation over an ontology K is a function $I : AP \cup \Sigma_E \rightarrow \mathcal{F}(S, F, R)$.

Intuitively, I assigns to each $p \in AP$ the domain condition that must hold at any state where p is true, and to each event $a \in \Sigma_E$ the domain condition that characterizes the occurrence of that event. In both cases, I acts as a bridge between the engineer's formal model and the physical world: it makes explicit what the engineer means when they write a TCTL property over p , or when the system fires event a , in terms of the entities, quantities, and constraints of the ontology K .

Consider again the example in Section II and the K previously defined. Let $\text{spraying} \in AP$ hold whenever the drone is actively dispensing pesticide, and let $\text{shutoff} \in \Sigma_E$ be the event fired when the drone halts to avoid battery depletion. An interpretation I of these labels over K could be: $I_P(\text{spraying}) = \neg \text{done}(\text{cur_hec}) \wedge \text{cons}(\text{cur_hec}) > 0$, capturing the state of active spraying, and $I_P(\text{shutoff}) = \text{tot_con} + \text{cons}(\text{cur_hec}) > \text{tot_bat}$, capturing the triggering condition for the shutdown event.

Finally, domain knowledge is then defined as $\Phi = (K, I)$, structuring the knowledge and its connection to the system's behavior, forming the shared semantic ground on which PT/DT alignment is established.

Since domain knowledge evolves as engineers discover new facts, refine existing ones, and incorporate updated standards, we must ensure that such evolution does not invalidate established alignment guarantees. To reason about this, we introduce an order on Φ instances.

Definition 4. Let $\Phi_1 = (K_1, I_1)$ and $\Phi_2 = (K_2, I_2)$. We say Φ_1 *refines* Φ_2 , written $\Phi_1 \sqsubseteq \Phi_2$, if:

- $(S_1, F_1, R_1) \supseteq (S_2, F_2, R_2)$,
- $\Delta_1 \models \Delta_2$
- $\Delta_1 \models I_1(p) \leftrightarrow I_2(p)$ for all $p \in AP$.

Condition a allows K_1 to introduce new value domains, functions, and relations not present in K_2 , e.g., adding a type `Motor` with a function $\text{rpm} : \text{Motor} \rightarrow \mathbb{R}$ once motor dynamics become relevant. Condition b requires that everything K_2 knew to be true is still entailed by K_1 , ensuring

that no prior knowledge is contradicted. Condition c requires that $I_1(p)$ and $I_2(p)$ are equivalent under Δ_1 . This ensures the two interpretations are consistent under the refined domain knowledge. For example, once Δ_1 asserts $\text{tot_con} \leq 450$, both $I_1(\text{bat_ok}) = \text{tot_con} \leq 450$ and $I_2(\text{bat_ok}) = \text{tot_con} \leq 500$ are equivalent under any model of Δ_1 .

Together, these three conditions ensure that Φ_1 is genuinely more informative than Φ_2 : it can reason about more concepts, its axioms are at least as strong, and its atomic propositions are at least as precisely defined. The same conservative pattern governs the evolution of safety-critical standards themselves: DO-178C extended DO-178B without removing objectives [48], IEC 61511 second edition tightened existing clauses without relaxing prior requirements [49], and FAO-56 Rev.1 updated crop coefficients based on accumulated field data [50]. In all cases, knowledge accumulates: new concepts are added, constraints are tightened and, specifically in the case of standards, nothing is removed or contradicted, exactly mirroring the structure formalized in $\sqsubseteq$.

VI. SEMANTIC ALIGNMENT BETWEEN PT AND DT

For DT-based reasoning to be correct, the DT must be semantically aligned with the PT. This requires two properties: (i) the twins agree on the meaning of observed data and share a consistent interpretation of the domain; (ii) they exhibit corresponding behavior in all aspects relevant to monitoring and decision-making.

We assume a shared ontology K as the semantic interface between the twins, from which two interpretations I_P and I_D are derived. Ontologies are widely used in DT applications [31], [32] and interoperability frameworks [16], [51], [52], [53] to provide a common semantic foundation for data exchange and reasoning, making this assumption well-grounded in practice.

Let $\mathcal{A}_P$ and $\mathcal{A}_D$ be the TAs representing the PT and DT views, respectively, with $\llbracket \mathcal{A}_P \rrbracket = (S^P, \Sigma_E^P, \Sigma_D^P, \rightarrow^P, L^P)$ and $\llbracket \mathcal{A}_D \rrbracket = (S^D, \Sigma_E^D, \Sigma_D^D, \rightarrow^D, L^D)$ be their semantic representations, and let $A = \text{dom}(I_P) \subseteq \Sigma_E^P$ and $B = \text{dom}(I_D) \subseteq \Sigma_E^D$ be their ontologically relevant sub-alphabets.

Definition 5. Semantic alignment is a relation $\mathcal{R} \subseteq S^P \times S^D$ such that $(s_0^{V_P}, s_0^{V_D}) \in \mathcal{R}$ and for every $(p, d) \in \mathcal{R}$:

- I) $\Delta \models I_P(L^P(p)) \leftrightarrow I_D(L^D(d))$ (state consistency)
- II) $\forall a \in A, p' \text{ with } p \xrightarrow{a} p', \exists a' \in B, d' : d \xrightarrow{a'} d' \wedge (p', d') \in \mathcal{R} \wedge \Delta \models I_P(a) \leftrightarrow I_D(a')$ (PT-to-DT)
- III) $\forall a' \in B, d' \text{ with } d \xrightarrow{a'} d', \exists a \in A, p' : p \xrightarrow{a} p' \wedge (p', d') \in \mathcal{R} \wedge \Delta \models I_P(a) \leftrightarrow I_D(a')$ (DT-to-PT)
- IV) $\forall \delta \in \Sigma_D^D, p' \text{ with } p \xrightarrow{\delta}^* p', \exists d' \text{ reachable from } d \text{ by } \xrightarrow{\delta}^* \text{ with elapsed time } \delta \text{ such that } (p', d') \in \mathcal{R}, \text{ and symmetrically for } \Sigma_D^P$ (delay consistency)

Definition 6. Weak semantic alignment is a relation $\mathcal{R} \subseteq S^P \times S^D$ such that $(s_0^{V_P}, s_0^{V_D}) \in \mathcal{R}$ and Conditions I, II and IV of Definition 5 are satisfied.

We write $V_P \stackrel{\Phi}{\sim} V_D$ for the greatest strong alignment relation and $V_P \preceq_{\Phi} V_D$ for its weak counterpart. Both

Algorithm 1: Semantic Alignment Check

Input: $\Gamma(\mathcal{A}_P), \Gamma(\mathcal{A}_D), \Phi = (K, \{I_P, I_D\})$
Output: True iff $\mathcal{A}_P \stackrel{\Phi}{\sim} \mathcal{A}_D$, else False

```

1  $E \leftarrow (a, a') \in A \times B \mid \Delta \models I_P(a) \leftrightarrow I_D(a')$ 
2  $\tilde{\Phi} \leftarrow (q_p, q_d) \in Q_P \times Q_D \mid \Delta \models I_P(L^P(q_p)) \leftrightarrow I_D(L^D(q_d))$ 
3 repeat
4    $\tilde{\Phi}' \leftarrow \tilde{\Phi}$ 
5   foreach  $(p, d) \in \tilde{\Phi}$  do
6     foreach  $a \in A, q_p \xrightarrow{a} q'_p$  do // Cond. II
7       if  $\nexists (a', q'_d) : (a, a') \in E, q_d \xrightarrow{a'} q'_d, (q'_p, q'_d) \in \tilde{\Phi}$ 
8         then  $\tilde{\Phi}' \leftarrow \tilde{\Phi}' \setminus (q_p, q_d)$ 
9     foreach  $a' \in B, q_d \xrightarrow{a'} q'_d$  do // Cond. III
10      if  $\nexists (a, q'_p) : (a, a') \in E, q_p \xrightarrow{a} q'_p, (q'_p, q'_d) \in \tilde{\Phi}$ 
11        then  $\tilde{\Phi}' \leftarrow \tilde{\Phi}' \setminus (p, d)$ 
12      foreach  $q_p \xrightarrow{\tau} q'_p$  do // Cond. IV
13        if  $\nexists q'_d : q_d \xrightarrow{\tau} q'_d, (q'_p, q'_d) \in \tilde{\Phi}$  then
14           $\tilde{\Phi}' \leftarrow \tilde{\Phi}' \setminus (q_p, q_d)$ 
15      foreach  $q_d \xrightarrow{\tau} q'_d$  do // Cond. IV symm.
16        if  $\nexists q'_p : q_p \xrightarrow{\tau} q'_p, (q'_p, q'_d) \in \tilde{\Phi}$  then
17           $\tilde{\Phi}' \leftarrow \tilde{\Phi}' \setminus (q_p, q_d)$ 
18    $\tilde{\Phi} \leftarrow \tilde{\Phi}'$ 
19 until  $\tilde{\Phi}$  stabilises
20 return True if  $(q_p^0, q_d^0) \in \tilde{\Phi}$ , False otherwise

```

are lifted to TAs via the TTSs they induce: $V_P \stackrel{\Phi}{\sim} V_D$ iff $\llbracket \mathcal{A}_P \rrbracket \stackrel{\Phi}{\sim} \llbracket \mathcal{A}_D \rrbracket$, and similarly for weak alignment.

Condition I ensures that matched states carry equivalent domain-level meaning under Φ : the PT and DT agree on the facts of the world at every point in their execution. Conditions II and III extend the standard bisimulation clauses by requiring ontologically equivalent labels to match, rather than syntactically identical ones. Labels outside A and B are treated as τ -transitions, invisible to the alignment. This lets the PT and DT include internal events irrelevant to the other, e.g. physical events on the PT side, logging or external API calls on the DT side, while remaining aligned on the observable behavior. Finally, Condition IV ensures timing is preserved across matched pairs.

To decide semantic alignment, we operate on the zone abstractions $\Gamma(\mathcal{A}_P)$ and $\Gamma(\mathcal{A}_D)$ with symbolic states Q_P and Q_D , finite abstractions that preserve the timed behavior of the underlying automata [29], [30]. Algorithm 1 follows the standard greatest fixpoint characterization of bisimulation [21], with two extensions that operationalize Definition 5 directly. First, the label equivalence relation E is precomputed once via SMT entailment checks over Φ (line 1), requiring at most $|A| \cdot |B|$ solver calls. Second, $\tilde{\Phi}$ is seeded with all symbolic state pairs satisfying state consistency (line 2). The loop then iteratively prunes pairs violating any of Conditions II–IV (omitting Condition III for weak alignment), consulting E for ontological label equivalence until stabilisation.

Theorem 1. *Algorithm 1 yields the largest relation $\overset{\Phi}{\sim}$.*

Theorem 1 follows from a standard greatest-fixpoint argument: the zone graph’s state space is finite [29], [30], and since the pruning operator is monotone ($\overset{\Phi'}{\sim} \subseteq \overset{\Phi}{\sim}$ at each iteration), the loop converges to the greatest fixpoint by Tarski’s theorem, at which point every surviving pair satisfies Definition 5 (hence Definition 6) by construction. Computing E costs $O(|A| \cdot |B| \cdot \gamma)$, where γ is the cost of an SMT call (e.g., polynomial for quantifier-free linear arithmetic [40]); each iteration visits at most $|Q_P| \cdot |Q_D|$ pairs against E , over at most $|Q_P| \cdot |Q_D|$ iterations, for a total worst-case cost of $O(|Q_P|^2 \cdot |Q_D|^2 \cdot (|A| + |B|) \cdot \gamma)$.

VII. GUARANTEES OF SEMANTIC ALIGNMENT

Semantic alignment is the fundamental correctness criterion of our framework. In this section, we investigate the guarantees that semantic alignment provides, and their practical implications for the design and use of DTs. For the rest of this section, fix a domain knowledge $\Phi = (K, \{I_P, I_D\})$ and two TAs V_P and V_D as PT and DT views, respectively.

a) *Aligned Domain Interpretation:* Semantic alignment guarantees that the PT and DT share a consistent interpretation of the domain. That is, at every matched state, any domain fact derivable from the PT state is equivalently derivable from the corresponding DT state, and vice versa.

Theorem 2. *Assume V_P and V_D are (weakly) semantically aligned. For every $(p, d) \in \overset{\Phi}{\sim}$ and every $f \in \mathcal{L}(K)$:*

$$\Delta \cup \{I_P(L^P(p))\} \models f \iff \Delta \cup \{I_D(L^D(d))\} \models f.$$

Proof. By Definition 5, Condition I, every $(p, d) \in \overset{\Phi}{\sim}$ satisfies $\Delta \models I_P(L^P(p)) \leftrightarrow I_D(L^D(d))$. Hence for every model $\mathcal{M} \models \Delta$ we have $\mathcal{M} \models I_P(L^P(p))$ iff $\mathcal{M} \models I_D(L^D(d))$, so the theories $\Delta \cup \{I_P(L^P(p))\}$ and $\Delta \cup \{I_D(L^D(d))\}$ have exactly the same class of models. Theories with the same models entail the same sentences; therefore $\Delta \cup \{I_P(L^P(p))\} \models f$ iff $\Delta \cup \{I_D(L^D(d))\} \models f$ for every $f \in \mathcal{L}(K)$. $\square$

This guarantee holds at design time across all reachable matched states. The agreement is not on raw data values, which the PT and DT may encode differently, nor on label vocabulary, which need not coincide in federated settings, but on the semantic content of the state of the world under Φ . In particular, any sensor reading or physical quantity expressible in $\mathcal{L}(K)$ that triggers a domain fact for the PT triggers the equivalent fact for the DT at the matched state, and vice versa. As we are assuming a shared ontology, any value outside the domain of K is irrelevant for alignment, and can be encoded differently by the twins without affecting their semantic agreement.

b) *Aligned Behavior:* The second guarantee concerns behavioral consistency: any requirement relative to the observable behavior verified on the PT automatically holds on the DT, and vice versa. We formalize this by showing that the PT and DT satisfy the same TCTL properties over their ontologically relevant alphabets.

To transfer properties, we identify which PT and DT labels carry the same domain-level meaning under Φ . Labels $a \in \text{dom}(I_P)$ and $b \in \text{dom}(I_D)$ are equivalent under Δ , written $a \equiv_\Delta b$, if $\Delta \models I_P(a) \leftrightarrow I_D(b)$; this groups each view’s labels into equivalence classes. The induced label translation $\lambda : \text{dom}(I_P)/\equiv_\Delta \rightarrow \text{dom}(I_D)/\equiv_\Delta$ defined by $\lambda([a]) = [a'] \iff \Delta \models I_P(a) \leftrightarrow I_D(a')$ then maps each equivalence class of PT labels to the corresponding class of DT labels, acting as a dictionary that bridges the two vocabularies. By Definition 5, λ covers all labels reachable under the alignment relation. For a TCTL property φ over $\text{dom}(I_P)$, $\lambda(\varphi)$ substitutes each $a \in \text{dom}(I_P)$ in φ with the canonical representative of $\lambda([a])$.

Theorem 3. *Let $V_P \overset{\Phi}{\sim} V_D$. For every TCTL formula φ over $\text{dom}(I_P)$: $\mathcal{A}_P \models \varphi \iff \mathcal{A}_D \models \lambda(\varphi)$ and symmetrically, for every ψ over $\text{dom}(I_D)$: $\mathcal{A}_D \models \psi \iff \mathcal{A}_P \models \lambda^{-1}(\psi)$*

Proof. λ is a well-defined partial injection on equivalence classes. If $\Delta \models I_P(a) \leftrightarrow I_D(a')$ and $\Delta \models I_P(a) \leftrightarrow I_D(a'')$, then $\Delta \models I_D(a') \leftrightarrow I_D(a'')$, i.e. $[a'] = [a'']$, so the image of $[a]$ is unique; and $a \equiv_\Delta b$ gives $\Delta \models I_P(a) \leftrightarrow I_P(b)$, so $[a]$ and $[b]$ share the same image, making $\lambda([a])$ representative-independent. Symmetrically, $\Delta \models I_P(a) \leftrightarrow I_D(a')$ and $\Delta \models I_P(b) \leftrightarrow I_D(a')$ force $[a] = [b]$, so λ is injective and restricts to a bijection between the classes reached under $\overset{\Phi}{\sim}$, with λ^{-1} well-defined on its range. Fix a canonical representative per class and let $\lambda(\mathcal{A}_D)$ be $\mathcal{A}_D$ with every transition relabelled by its class representative. By Conditions II and III, every matched transition $p \xrightarrow{a} p'$ is mirrored by $d \xrightarrow{a'} d'$ with $[a'] = \lambda([a])$ and $(p', d') \in \overset{\Phi}{\sim}$, and conversely; after relabelling these carry identical labels, and Condition IV supplies the delay clause, so $\overset{\Phi}{\sim}$ is a syntactic WTB between $\mathcal{A}_P$ and $\lambda(\mathcal{A}_D)$ over $\text{dom}(\lambda)$. Since all labels in a class are Δ -equivalent, relabelling preserves denotation and leaves unchanged which domain events φ references. TCTL preservation under weak timed bisimulation [21] then gives $\mathcal{A}_P \models \varphi \iff \lambda(\mathcal{A}_D) \models \varphi \iff \mathcal{A}_D \models \lambda(\varphi)$; the claim for ψ over $\text{dom}(I_D)$ follows by the same argument applied to λ^{-1} . $\square$

Theorem 3 makes strong alignment practically significant: PT and DT views are behaviorally equivalent, so any TCTL property established on one transfers soundly to the other through λ , and the DT can serve as a behavioral surrogate for the PT in formal verification, predictive reasoning, and decision support. For example, an updated energy-efficient spray pattern can be validated on the DT by model-checking a TCTL reachability property—that a target hectare index is cleared within a timing bound without depleting the power budget—before deployment; because strong alignment preserves all branching-time semantics, the physical drone is guaranteed to afford exactly the behavior needed to replicate the optimization safely in the field.

Under weak alignment, by contrast, the DT may exhibit behaviors absent from the PT specification: the same property may hold on the DT yet fail on the PT, via a sequence of

states and transitions the PT does not support. Weak alignment thus remains sufficient for runtime monitoring and anomaly detection—where the DT need only observe and match every PT behavior—and offers more design flexibility, but strong alignment is strictly necessary for proactive, predictive reasoning over future trajectories to remain faithful to the PT.

By definition of λ , the framework permits many-to-many label mappings for generability and flexibility: a single PT label may map to multiple semantically equivalent DT labels, any one of which suffices for alignment. In safety-critical deployments where each PT event must have a unique DT counterpart, requiring $|\lambda([a])| = 1$ for all $a \in A_{safe} \subseteq A$ is a sufficient condition for bisimulation-criticality of every safety-critical label; it is decidable by the same SMT machinery used to precompute λ and can be enforced as a pre-condition to alignment verification at no additional cost.

c) Aligned Implementations: As established in Section III, PT and DT implementations must refine their respective views. A practical advantage of adopting TAs as the modeling formalism for the views is that they are directly amenable to automatic implementation synthesis [54], e.g. [55], [56], [57], [58]. This establishes a direct refinement path from verified views to deployed software. The DT, being software under the engineer’s control, can be constructed to exactly realize its views, formally $D \sim V_D$. The deployed PT, being a physical artifact, cannot reproduce every abstract behavior of its views but must not violate them, formally $P \preceq V_P$. Consequently, by the refinement chain $P \preceq V_P \stackrel{\Phi}{\sim} V_D \sim D$, the deployed artifacts inherit the guarantees established at the view level. In particular, both implementations are guaranteed to agree on the meaning of domain facts at every reachable aligned state, and any TCTL safety property verified on D soundly transfers to P . Therefore, any operational decision or boundary enforced by the DT is guaranteed to preserve safe behavior within the physical twin’s actual execution.

d) Alignment through evolution: Theorems 2 and 3 support independent evolution of the twins while preserving alignment. Since internal events are abstracted away by the alignment relation, PT and DT views can be updated independently, e.g., by adding internal transitions or refactoring structure, provided the observable behavior remains intact.

Particularly relevant in federated settings, establishing alignment at the view level lets implementations be developed, deployed, and maintained independently, without the coordination overhead or security risks of a shared runtime infrastructure. The sole shared artifact is the common ontology K ; each twin keeps its own private interpretation function, so interoperability is guaranteed by design-time alignment. Stakeholders can thus evolve their twin implementations over time, requiring only local re-verification of their views when internal software changes occur.

Furthermore, as domain knowledge evolves and gets refined during system development or deployment, the design-time guarantees of semantic alignment are preserved, as we establish in the following theorem.

Theorem 4. *Let Φ' and Φ be two domain knowledge instances such that $\Phi' \sqsubseteq \Phi$, if $V_P \stackrel{\Phi}{\sim} V_D$ then $V_P \stackrel{\Phi'}{\sim} V_D$.*

Proof. By Definition 5, alignment requires that for every matched pair $(a, a') \in A \times B$, $\Delta \models I_P(a) \leftrightarrow I_D(a')$, and for every matched state pair $(p, d) \in \Phi$, $\Delta \models I_P(L^P(p)) \leftrightarrow I_D(L^D(d))$. Since $\Phi' \sqsubseteq \Phi$, by condition b we have $\Delta' \models \Delta$, and by condition c we have $\Delta' \models I'_P(L^P(p)) \leftrightarrow I_P(L^P(p))$ and $\Delta' \models I'_D(L^D(d)) \leftrightarrow I_D(L^D(d))$ for all $p \in AP$. Hence, $\Delta' \models I'_P(a) \leftrightarrow I'_D(a')$ and $\Delta' \models I'_P(L^P(p)) \leftrightarrow I'_D(L^D(d))$. Therefore $\stackrel{\Phi'}{\sim}$ satisfies Definition 5. $\square$

Note that Theorem 4 holds for weak alignment as well.

This supports iterative development and long-term lifecycle management: teams can refine Φ without re-aligning or re-verifying the cross-twin interfaces from scratch, so the deployed twins remain semantically interoperable as the domain knowledge matures.

VIII. EVALUATION

To assess our framework, we structure our evaluation around the following research questions:

- RQ1:** How does semantic alignment compare to the state of the art in DT/PT alignment?
- RQ2:** What is the difference between syntactic and semantic fidelity in DT alignment?
- RQ3:** Is the diagnostic output sufficient to localise and repair faults without inspecting the automaton structure?
- RQ4:** What drives algorithm runtime in practice?

Section VIII-A addresses **RQ1** and **RQ2** through a real-world case study. **RQ3** and **RQ4** are addressed in Section VIII-B, where we conducted an experiment on a suite of eight case studies of varying size and complexity.

A. Case Study: DT for Autonomous Crop Spraying

This case study arises from our own experience developing the energy management DT in the scenario of Section II.

a) Scenario Description: A fleet of drones, each equipped with a 355 Wh battery pack, is tasked with autonomous crop-spraying missions over a nine-hectare field. The DT maintains a representation of the drone and the field, using it to derive a strategy that maximises field coverage while avoiding battery depletion. The field is represented as a nine-cell grid, with each cell’s crop density and atmospheric conditions determining its energy requirements, ranging nominally from 10 to 30 Wh. The return cost is 10 Wh. Under high humidity and low atmospheric pressure, the drone’s Battery Management System (BMS) reports a battery stress event characterized by elevated internal resistance, which increases per-cell consumption by 45 Wh.

b) Engineering Process: Our first DT modelled energy consumption under nominal conditions, under which it performed correctly; the fault manifested only when stress occurred at a late field cell, resulting in battery depletion mid-mission (**Stage 1**). The canonical engineering response was to equip the DT with a reactive safety monitor [59], [60]

observing PT battery telemetry directly, firing when the battery could no longer safely complete the current cell and return to base (**Stage 2**). The monitor eliminated depletions but, because it fires after the drone has already committed to a cell it cannot safely complete, forced mid-spray aborts whenever stress was detected too late, proving operationally unacceptable. Specifically, the mission completion rate dropped and the aborted cells failed to meet the FAO-56 minimum pesticide application rate, a violation the compliance auditor’s DT would flag as non-compliant. To address this, we developed and instantiated our framework (**Stage 3**), which enforced design-time strong semantic alignment between the DT and PT, ensuring that the DT’s behavior was consistent with the PT’s under all conditions, including stress. To mitigate the battery stress overhead, the aligned DT instructs the drone to increase ground speed and pump flow rate proportionally, reducing hover time per cell and thereby lowering resistive energy losses. Pesticide application becomes less precise but remains within FAO-56 minimum rate tolerances.

c) *Experimental Design*: We evaluate all three stages through simulation, running 5,000 missions per battery stress probability level across six levels (0%–50%), for a total of 30,000 simulated missions per scenario. The battery stress probability is varied to model a range of seasonal and meteorological conditions; at 0% all three scenarios are equivalent, establishing a shared nominal baseline. In each mission, the stress event is assigned to a cell drawn uniformly at random, modelling the unpredictability of its onset. Both DTs receive identical PT battery telemetry at every cell boundary. We measure battery depletion rate, mid-spray abort rate, mean cells completed per mission, and battery level at return.

d) *RQ1: Comparison to Existing Approaches*: Table I summarises the results aggregated across all battery stress probability levels. Stage 1 achieves 85.5% mission completion, i.e., all 9 cells are sprayed and the drone returns to base. In ~10% of missions, the DT correctly aborts before committing to a cell it cannot safely complete. However, more than 4% of missions end in battery depletion. These are precisely the cases where battery stress occurs late enough to fall within the DT’s unaligned safety margin, rendering the fault undetectable until commitment. Stage 2 eliminates depletions entirely, but introduces a new failure mode: the reactive monitor fires after the drone has already committed to a cell, forcing a mid-spray abort in 25.1% of missions. The operational cost is significant, as mean field coverage drops from 8.68 to 8.43 cells and mission completion falls to 74.9%. Stage 3 eliminates both failure modes: no depletions, no mid-spray aborts, and the highest coverage across all scenarios at 8.80 cells and 85.6%. Compared to Stage 1 and Stage 2, the average battery level at return of Stage 3 reflects a more optimized energy budget: lower than Stage 2’s conservative 90.9 Wh, suggesting it doesn’t waste coverage potential by firing too early, yet safely above depletion, unlike Stage 1’s 81.6 Wh which is partially driven down by depleted missions.

e) *Representative Mission Analysis*: Figure 2 shows two representative missions, namely **Mission 1** and **Mission 2**,

TABLE I: Scenario outcomes overview.

Scenario	Mission completed	Cells completed	Avg battery at return	Safe early returns	Mid-spray aborts	Battery depletions
Stage 1	85.5%	8.68	81.6 Wh	10.1%	0.0%	4.4%
Stage 2	74.9%	8.43	90.9 Wh	25.1%	25.1%	0.0%
Stage 3	85.6%	8.8	85.6 Wh	14.4%	0.0%	0.0%

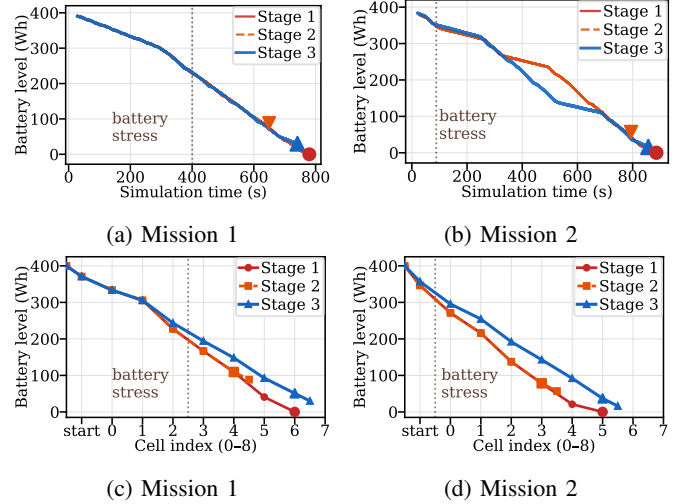


Fig. 2: Battery level during Mission 1 and Mission 2

illustrating the difference between the three stages. The two missions are identical in all respects except for the timing of the battery stress onset, which occurs after cell 2 in Mission 1 and before starting cell 0 in Mission 2. The former is a mid-stress mission, where the DT has already committed to several cells before stress occurs, while the latter is an early-stress mission, where the DT has only committed to a few cells before stress occurs.

Both missions share a similar pattern. The three stages follow identical battery profiles until the battery stress event onset, marked by the vertical dotted line (after ~500s in Mission 1 and after ~100s in Mission 2). After the onset, Stage 1 continues to follow the nominal profile, which underestimates consumption and leads to depletion in both missions. Specifically, the DT commits to a cell, which completes, but the battery is too depleted to return, stranding the drone mid-field. Stage 2 prevents depletion by firing the monitor mid-spray, predicting the drone would not have enough battery to complete the current cell and return. The mid-spray aborts occur at cell 4 in Mission 1 and cell 3 in Mission 2, which will be flagged as non-compliant by the auditor’s DT. Stage 3 resolves both failure modes by activating the speed boost immediately upon detecting stress, completing one additional cell and returning safely in both missions. The aligned DT’s battery profile reflects the difference in strategy. Stage 3’s battery level at each cell boundary remains consistently higher than Stage 1 and Stage 2 after stress onset. This is a direct consequence of the speed boost: by increasing ground speed and pump flow rate, the drone reduces hover time per cell, lowering total resistive energy losses. While Figure 2a may

suggest higher consumption for Stage 3 due to its steeper battery slope over time, Figure 2c clarifies that this reflects higher throughput, i.e., more cells completed per unit time, rather than higher energy consumption per cell.

f) **RQ2: Syntactic vs Semantic Alignment:** To address **RQ2**, we implemented a fidelity monitor inspired by [61], measuring the divergence between PT battery and DT predictions for Stage 1 and Stage 3 (Stage 2 reuses the same DT as Stage 1, adding only the reactive monitor, so its prediction gap is identical). We compared the results for the representative missions, reported in Table II. This comparison reveals that neither twin perfectly tracks the PT, with both exhibiting numerical divergence throughout, with no consistent separation between them. In Mission 1, DT1 and DT3 show indistinguishable prediction errors before battery stress onset. Only after the onset does the prediction gap widen for DT1, while DT3’s gap fluctuates non-monotonically, at times narrowing and at times exceeding DT1’s. Similarly, in Mission 2 the gap fluctuates and at times DT3’s error exceeds DT1’s.

We confirm this finding holds at scale. Across all 30,000 missions per stress-probability level, DT3 shows a small average improvement over DT1 (9.46Wh vs. 10.49Wh, as shown in Figure 3a) and this advantage grows with stress probability, from parity at 0% to a $\sim 2\%$ gap at 50% (Figure 3b), consistent with DT3 explicitly modeling the stress mechanism DT1 ignores. This advantage is nonetheless modest relative to the spread of both distributions: the interquartile ranges (IQR) overlap almost entirely ([5.3, 11.9]Wh for DT1 vs. [5.3, 11.2]Wh for DT3), and, as Mission 1 and Mission 2 exemplify, DT3’s error frequently exceeds DT1’s at the level of individual cells, with no discernible pattern tied to the onset of battery stress or to either twin’s alignment status.

This exposes a structural limitation of any syntactic alignment approach. A syntactic metric reports how divergent the DT’s predictions are from the PT’s, but does not assign meaning to this divergence. Two states with a 40Wh discrepancy may agree on every domain fact in $\mathcal{L}(\mathcal{K})$ relevant to the system’s correctness, e.g. both twins derive `bat_ok` and neither predicts `shutdown`, making the discrepancy inert. Conversely, a 2Wh discrepancy may result in different outcomes, with one twin deriving a fact (e.g. `tot_con > tot_bat`) that the other does not, making that small gap safety-critical. Contrary to syntactic approaches, semantic alignment does assign meaning to the observed divergence, and Theorem 2 ensures both twins share the same interpretation of it, making it possible for the DT to correctly predict the PT’s behavior even when the numerical gap is large.

B. Controlled study

The theoretical guarantees of Section VII establish correctness by construction; the following experiments characterize practical applicability by investigating **RQ3** and **RQ4**.

a) **Experimental Setup:** We implemented Algorithm 1 in C++17 (GCC 11.4) using Z3 [45] as the SMT solver. All experiments were performed on an Ubuntu-based machine with 8 vCPUs (AMD EPYC 7452 32-Core Processor) and

TABLE II: Gap to PT in Wh per cell

Cell	Mission 1			Mission 2		
	DT1	DT3	DT1 - DT3	DT1	DT3	DT1 - DT3
0	9.96	9.51	0.45	45.81	-2.30	43.51
1	-7.15	-7.87	-0.72	19.38	-28.16	-8.78
2	49.86	33.47	16.39	49.26	0.44	48.82
3	22.17	-23.63	-1.46	20.06	-20.56	-0.50
4	24.91	-21.85	3.06	25.65	-15.18	10.47
5	39.60	-8.61	30.99	-9.71	-7.16	2.55
6	-	-27.75	-	-	-46.61	-
7	-	-41.48	-	-	-	-

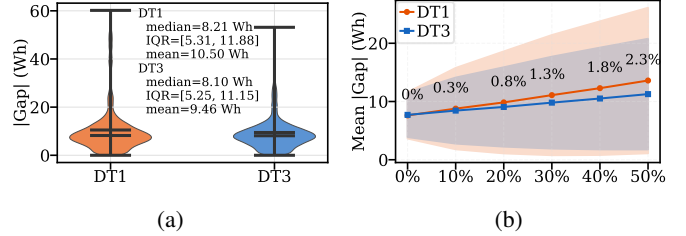


Fig. 3: Gap between PT and DT over all simulations.

125 GiB of RAM. The benchmark comprises eight case studies (Table III) drawn from safety-critical domains spanning process control, aerospace, medical devices, and precision agriculture. Ontologies are constructed from the governing standards for each domain. Reported times are the median wall-clock time in milliseconds across three runs.

b) **RQ3: Diagnosability of Semantic Misalignment:** Being based on WTB, our framework provides a precise witness when misalignment is detected: the label pair at which the fixpoint collapses identifies the behavioral difference between the two systems. Our framework adds a semantic dimension to this: because labels are mapped through I_P and I_D , the counterexample is not a logical formula over automaton states as in standard approaches [63], but a concrete valuation in $\mathcal{L}(\mathcal{K})$ directly interpretable by engineers without formal methods expertise. We illustrate this for two classes of injected faults.

In CS7, the semantic interpretation of the DT event representing the pilot authority override was deliberately omitted. Without this mapping, the DT event can not be proven equivalent to the PT’s event responsible for emergency override. The event is hence demoted to a τ -transition, excluded from E , and the fixpoint collapses at the first product state where the PT fires the override. From the witness pair, engineers can immediately identify the missing interpretation as the root cause, and repair the fault, after which alignment is restored. This is representative of a common class of faults, such as overconstrained or underconstrained interpretation formulas, that arise during the construction of the interpretations.

In CS8, we injected a fault in the form of a domain knowledge update that deliberately violates Condition c. The threshold to raise the irrigation alarm tightens from 15 to 13 mm under drought management provisions in FAO-56, but the DT interpretation retains the threshold of 20 mm.

TABLE III: Benchmark overview: CS1–CS8. Variations refers to the number of PT/DT pairs evaluated for each case study. $|\mathcal{A}_P| / |\mathcal{A}_D|$ = number of zones in the largest PT / DT zone graph. SMT = maximum number of $|\text{SMT}|$ calls across all variants.

Case Study Description	Ontology Standard	Axioms	$ \text{dom}(\mathcal{I}_P) $	$ \text{dom}(\mathcal{I}_D) $	Var.	$ \mathcal{A}_P $	$ \mathcal{A}_D $	$ \mathcal{A}_P \times \mathcal{A}_D $	$ \text{SMT} $
CS1: Three-tank liquid process control [31]	IEC 61511-1	35	20	20	3	1 000	1 347	1.347×10^6	180
CS2: Overhead travelling crane [62]	EN 13001-1	31	21	19	4	15 153	39 152	5.933×10^8	178
CS3: Autonomous inspection rover	NASA-STD-8739.8A	31	17	16	4	10 314	124 898	1.288×10^9	70
CS4: Aircraft engine controller	DO-178C	26	23	21	4	182	132	2.402×10^4	253
CS5: Robotic debris grasping arm	ISO 9283	21	20	20	4	43	60	2.580×10^3	308
CS6: GPCA infusion pump	IEC 60601-1	16	21	21	4	12 052	3 922	4.727×10^7	208
CS7: Autopilot flight safety system	DO-178C / ARP4754A	22	27	24	5	2 576	1 462	3.766×10^6	506
CS8: Precision crop irrigation system	ISO 11783-7 / FAO-56	33	19	19	3	303 531	47 055	1.428×10^{10}	154

Condition c is violated, as $\Delta' \not\models I_P(\text{def_alarm}) \leftrightarrow I_D(\text{def_thresh_breach})$. Algorithm 1 reports the misalignment pair. The diagnostic output is directly interpretable by domain engineers: the equivalence $\Delta' \models (\text{deficit} > 13) \leftrightarrow (\text{deficit} > 20)$ produces the countermodel $\text{deficit} = 14$: a concrete agronomic value at which the PT triggers irrigation but the DT does not, corresponding to the crop water stress zone defined in FAO-56. This shows that when an ontology update breaks alignment the framework not only detects the violation but immediately localises it to the offending interpretation formula, enabling a targeted repair without any inspection of the automaton structure.

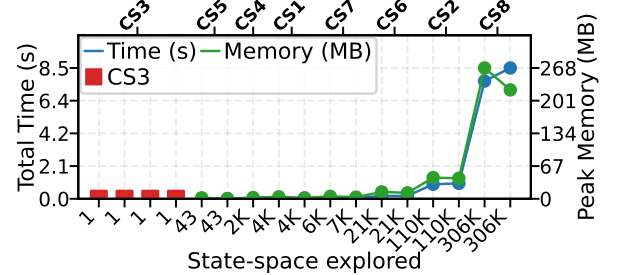
Finally, we included a degenerate case in CS3, where we constructed an interpretation in which the DT merges multiple safety-critical PT events into a single equivalence class under λ . Algorithm 1 reports alignment, stabilizing immediately as the fixpoint relation has size 1 (Figure 4). This is precisely the degenerate case identified in Section VII. Introducing the condition $|\lambda([a])| = 1$ for all $a \in A_{\text{safe}}$ in the precomputation step of Algorithm 1 would flag this case immediately, confirming the fault at no additional cost.

c) RQ4: Tractability: The case studies span three orders of magnitude in zone graph size, with the theoretical upper bound shown in Table III. Algorithm 1 completes all alignment checks within 7.6 s and 269 MB of peak memory, remaining tractable throughout. Both time and memory scale with $|\Phi|$ rather than the product state space, because the seeding step (line 2) discards the majority of candidate pairs via a single SMT-checked condition per pair before the iterative pruning loop ever runs: across all case studies, peak memory averages approximately 0.9 KB per surviving pair, and CS8, despite its 14.3 billion-pair product, seeds and converges to a fixpoint relation of only 305,697 pairs, completing in 7.6 s and 269 MB. The SMT precomputation cost for E is independent of zone graph size, bounded by $|\mathcal{A}| \times |\mathcal{B}|$ solver calls; actual calls remain consistently between 26% and 78% of this bound, with higher utilisation in case studies whose timing variants introduce additional state-pair seeding.

C. Threats to Validity

a) Internal Validity: Our **RQ1** and **RQ2** results reflect our instantiation of the framework in a real-world deployment scenario and are not meant to be read as claims for general bounds. What generalizes is the qualitative difference. For

Fig. 4: Total Time and Memory Usage over CS1 to CS8



RQ1 what generalizes is the source of the improvement: Stage 1’s depletions and Stage 3’s mid-spray aborts both stem from a behaviour that fails to correspond to the PT, the failure mode semantic alignment rules out by construction (Theorems 2-3). Stage 3 removes this misalignment rather than its symptoms, a distinction independent of the specific numbers. Similarly, **RQ2**’s numbers illustrate, on a concrete instance, that syntactic divergence and semantic correctness are orthogonal, a distinction that holds regardless of the magnitude of divergence observed in any particular system.

b) External Validity: The case studies of Section VIII-B were deliberately constructed across diverse domains to exercise the framework broadly and to support fault injection with known ground truth for **RQ3**, chosen as representative of common misalignment classes. Naturally occurring misalignments may combine fault classes beyond our taxonomy, but the diagnostic mechanism itself is fault-agnostic, as it depends only on where the fixpoint computation fails. Regarding **RQ4**, our results characterize Algorithm 1’s behavior but do not constitute a full scalability analysis (zone-graph bisimulation [30] and Z3 [45] are independently well-studied). What **RQ4** shows is that cost tracks $|\Phi|$, remaining tractable even when the state-space is on the order of billions. A broader, community-contributed benchmark of industrial ontologies and views would help characterize what properties of Φ govern this gap.

c) Construct Validity: Definition 5 is instantiated over TTS and WTB, motivated by TCTL preservation and the flexibility to abstract internal, non-ontological behavior. The definition and its operationalization itself, however, is not tied to this choice: its extension to other formalisms (e.g., untimed labeled transition systems) is straightforward, and the

underlying relation can be replaced by strong bisimulation or other equivalences/preorders as required by the target setting, without altering the structure of Definition 5 itself.

IX. RELATED WORK

We evaluate prior work on PT/DT alignment along three dimensions: semantic interoperability, trace monitoring, and formal behavioral verification.

Ontologies are widely deployed in DT architectures for knowledge representation and interoperability [32]. Dominant approaches employ ontologies to map telemetry data to domain concepts for data-level interoperability [33], [64], or to describe DT capabilities and services to enable cross-organizational discovery and orchestration [65], [66], [67], [68]. Similarly, federated architectures such as Gaia-X [16] and Industry 4.0 frameworks [51], [52], [69] rely on shared ontologies and standardized interfaces to achieve interoperability across organizational boundaries. Kamburjan et al. [62] utilize semantic lifting into knowledge graphs to detect runtime structural drift, but leave behavioral correspondence unformalized as simple data-stream threshold monitoring. While these frameworks operate at the data-definition layer—structuring and exchanging knowledge to enable conceptual interoperability—they fail to formally relate the observable behaviors of the two twins. Independently developed twins can thus conform to the same static ontology while exhibiting divergent or contradictory physical dynamics. In contrast, our framework leverages the ontology as a formal semantic ground to guarantee both data interpretation and behavioral equivalence over all possible executions.

PT/DT alignment is frequently addressed via runtime trace monitoring. Muñoz et al. [61] check correspondence by syntactically aligning execution traces of PT and DT. Zhang et al. [70] propose knowledge-equivalence checking for DTs of learning agents, comparing the weighted interaction graphs maintained by PT and DT at runtime and triggering re-synchronization when their drift exceeds a threshold. Huang et al. [71] use Temporal Logic of Actions to formally specify and verify synchronization between PT and DT data flows (e.g., non-stale commands). Hallé et al. [60] detect divergence by comparing output traces against formal properties. DTChecker [72] synthesizes Signal Temporal Logic (STL) monitors to check compliance over live data streams.

These approaches suffer from three fundamental limits. First, they operate syntactically, addressing neither the semantic interpretation of the data—missing alignment when twins use different vocabularies—nor the equivalence of observable behaviors in the semantic space. Second, they require designing, implementing, and maintaining explicit runtime monitors for a fixed set of properties, an overhead that scales with property count and demands manual re-validation as the system evolves. Third, bound to observed historical executions, they cannot reason about future behaviours. This motivates TCTL: linear temporal logics like STL lack branching-time quantification and so cannot express the reachability-style properties needed for proactive, look-ahead planning.

To enable predictive reasoning, Seok et al. [59] use Timed Petri Nets as DTs to predict the PT’s next observable event via reachability analysis, checking consistency syntactically at runtime. However, these predictions remain unverified hypotheses until confirmed reactively by runtime observation of the actual PT traces. Under our framework (Section III), alignment is guaranteed by design without run-time infrastructure, making their runtime check redundant.

Alignment is also addressed through co-simulation frameworks that couple PT and DT data streams [73], [74]. Beyond the reactive limitations shared with monitoring approaches, co-simulation requires a runtime executable DT model continuously coupled to the PT, an infrastructure and integration cost our framework avoids entirely. Moreover, synchronization establishes neither agreement on the meaning of the data nor correspondence of observable events, and does not accommodate twins expressed over different vocabularies, as required in federated settings.

Our framework addresses these limitations by construction at design time. Rather than evaluating alignment reactively, we establish formal semantic alignment over all reachable behaviors of both twins, guaranteeing data-interpretation and behavioral equivalence and enabling proactive reasoning about future behaviors, without runtime monitoring infrastructure. Alignment is preserved across updates to the twins and their shared ontology without re-verification, shifting the engineering cost from continuous runtime tracking to a one-time design effort. Monitoring nevertheless remains complementary for detecting deployment-time deviations [75] or quantifying uncertainty [76], outside our scope.

Taken together, whereas prior work separately addresses ontology-based interoperability and runtime monitoring, ours is the first to define semantic alignment between PT and DT as a behavioral equivalence under a shared ontology, established by construction at design time and preserved across ontology evolution.

X. CONCLUSION

We presented a formal software engineering framework for establishing and preserving alignment between PT and DT. Based on the first formal definition of semantic alignment, the framework guarantees consistency in data interpretation and equivalence of observable behaviors, established at design time, inherited by implementations, and preserved across ontology evolution. We evaluated it on a real-world drone-spraying case study and eight use cases, showing it outperforms the state of the art, supports misalignment diagnosability and remains tractable on models with billions of states.

As future work, we plan to investigate how the design-time guarantees can be complemented to detect deviations caused by phenomena outside the shared ontology (e.g., undocumented degradation modes). We also plan to study whether the coverage condition for safety-critical labels should be folded into the alignment definition itself, and to establish a broader, community-contributed benchmark of ontologies and views to further characterize the scalability of our approach.

REFERENCES

- [1] INCOSE, *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, 5th ed., D. D. Walden, G. J. Roedler, K. J. Forsberg, R. D. Hamelin, and T. M. Shortell, Eds. Hoboken, NJ, USA: John Wiley & Sons, 2023.
- [2] S. Kugele, P. Obergfell, and E. Sax, “Model-based resource analysis and synthesis of service-oriented automotive software architectures,” *Software and Systems Modeling*, vol. 20, no. 6, pp. 1945–1975, 2021.
- [3] M. Broy, W. Damm, S. Henkler, K. Pohl, A. Vogelsang, and T. Weyer, “Introduction to the spes modeling framework,” in *Model-Based Engineering of Embedded Systems: The SPES 2020 Methodology*. Springer, 2012, pp. 31–49.
- [4] EUROCAE Working Group 71, *Software Considerations in Airborne Systems and Equipment Certification*, EUROCAE Std. ED-12C / DO-178C, 2011, european equivalent of DO-178C, harmonized standard for avionics software certification.
- [5] RTCA Special Committee 205, *Software Considerations in Airborne Systems and Equipment Certification*, RTCA, Inc. Std. DO-178C / ED-12C, 2011, issued jointly with EUROCAE as ED-12C; guidance for development of safety-critical airborne software.
- [6] ISO/IEC/IEEE, *Systems and Software Engineering — Architecture Description*, ISO/IEC/IEEE Standard 42010, 2011.
- [7] M. Grieves, “Digital twin: Manufacturing excellence through virtual factory replication,” University of Michigan, Tech. Rep., 2005, available online at <https://doi.org/10.13140/RG.2.2.26367.02729>.
- [8] W. Kritzinger, M. Karner, G. Traar, J. Henjes, and W. Sihn, “Digital twin in manufacturing: A categorical literature review and classification,” *IFAC-PapersOnLine*, vol. 51, no. 11, pp. 1016–1022, 2018.
- [9] F. Tao, Q. Qi, A. Liu, and A. Kusiak, “Data-driven smart manufacturing,” *Journal of Manufacturing Systems*, vol. 48, pp. 157–169, 2018.
- [10] R. Minerva, G. M. Lee, and N. Crespi, “Digital twin in the IoT context: A survey on technical features, scenarios, and architectural models,” *Proceedings of the IEEE*, vol. 108, no. 10, pp. 1785–1824, 2020.
- [11] D. M. Botín-Sanabria, A.-S. Mihaita, R. E. Peimbert-García, M. A. Ramírez-Moreno, R. A. Ramírez-Mendoza, and J. d. J. Lozoya-Santos, “Digital twin technology challenges and applications: A comprehensive review,” *Remote Sensing*, vol. 14, no. 6, p. 1335, 2022.
- [12] S. Mihai, M. Yaqoob, D. V. Hung, W. Davis, P. Towakel, M. Raza, M. Karamanoglu, B. Barn, D. Shetve, R. V. Prasad *et al.*, “Digital twins: A survey on enabling technologies, challenges, trends and future prospects,” *IEEE Communications Surveys & Tutorials*, vol. 24, no. 4, pp. 2255–2291, 2022.
- [13] F. Tao, M. Zhang, Y. Liu, and A. Y. C. Nee, “Digital twin driven prognostics and health management for complex equipment,” *CIRP Annals*, vol. 67, no. 1, pp. 169–172, 2018.
- [14] J. Mertens, S. Klikovits, F. Bordeleau, J. Denil, and Ø. Haugen, “Continuous evolution of digital twins using the DarTwin notation,” *Software and Systems Modeling*, 2024.
- [15] D. Lehner, J. Zhang, J. Pfeiffer, S. Sint, A.-K. Splettstößer, M. Wimmer, and A. Wortmann, “Model-driven engineering for digital twins: a systematic mapping study,” *Software and Systems Modeling*, 2025.
- [16] GAIA-X European Association for Data and Cloud AISBL, “Gaia-x architecture document,” https://gaia-x.eu/pdf/Gaia-X_Architecture_Document_2103.pdf, 2021.
- [17] E. B. Johnsen, E. Kamburjan, A. Pferscher, and S. L. Tapia Tarifa, “Formal methods meet digital twins: Challenges and opportunities,” in *Programming Languages and Systems*, R. Krebbers, Ed. Cham: Springer Nature Switzerland, 2026, pp. 1–12.
- [18] N. Rozanski and E. Woods, *Software Systems Architecture: Working with Stakeholders Using Viewpoints and Perspectives*, 2nd ed. Addison-Wesley, 2011.
- [19] P. Kruchten, “Architectural blueprints—the “4+1” view model of software architecture,” *IEEE Software*, vol. 12, no. 6, pp. 42–50, 1995.
- [20] R. Alur and D. L. Dill, “A theory of timed automata,” *Theoretical Computer Science*, vol. 126, no. 2, pp. 183–235, 1994. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/0304397594900108>
- [21] C. Baier and J.-P. Katoen, *Principles of Model Checking*. MIT Press, 2008.
- [22] J. Bengtsson, K. Larsen, F. Larsson, P. Pettersson, and W. Yi, “Up-paal—a tool suite for automatic verification of real-time systems,” in *Proceedings of the DIMACS/SYCON Workshop on Hybrid Systems III: Verification and Control: Verification and Control*. Berlin, Heidelberg: Springer-Verlag, 1996, p. 232–243.
- [23] X. Guo, H.-H. Lin, K. Yatake, and T. Aoki, “An UPPAAL framework for model checking automotive systems with FlexRay protocol,” in *Formal Techniques for Safety-Critical Systems (FTSCS 2013)*, ser. Communications in Computer and Information Science, vol. 419. Springer, 2014, pp. 51–66.
- [24] P. L. Laursen, V. A. T. Trinh, and A. E. Haxthausen, “Formal modelling and verification of a distributed railway interlocking system using UPPAAL,” in *Leveraging Applications of Formal Methods, Verification and Validation: Applications (ISoLA 2020)*, ser. Lecture Notes in Computer Science, vol. 12478. Springer, 2020, pp. 415–433.
- [25] M. Mikučionis, K. G. Larsen, J. I. Rasmussen, B. Nielsen, A. Skou, S. U. Palm, J. S. Pedersen, and P. Hougard, “Schedulability analysis using Uppaal: Herschel-Planck case study,” in *Leveraging Applications of Formal Methods, Verification, and Validation (ISoLA 2010)*, ser. Lecture Notes in Computer Science, vol. 6416. Springer, 2010, pp. 175–190.
- [26] S. Henkler and M. Hirsch, “Contract-based digital twin synthesis for autonomous safety critical systems,” in *Proceedings of the 56th Hawaii International Conference on System Sciences (HICSS-56)*. University of Hawaii at Manoa, 2023, pp. 6778–6787. [Online]. Available: <https://hdl.handle.net/10125/102456>
- [27] M. Naem and C. Seceleanu, “Contract-based verification of digital twins,” in *Engineering of Complex Computer Systems*, Y. Zhou, S. G. Teo, X. Xie, Z. Ding, and Y. Liu, Eds. Cham: Springer Nature Switzerland, 2026, pp. 338–357.
- [28] R. Gunasekaran and B. R. Haverkort, “Verification of digital twins using classical and statistical model checking,” in *Proceedings of the First International Workshop on Autonomous Systems Quality Assurance and Prediction with Digital Twins, ASQAP 2025, Hamilton, Canada, 4th May 2025*, ser. EPTCS, M. Chechik, A. Fedeli, G. Filippone, F. Formica, M. Frasheri, N. Hochgeschwender, and L. Marso, Eds., vol. 418, May 2025, pp. 16–23. [Online]. Available: <https://doi.org/10.4204/EPTCS.418.2>
- [29] R. Alur, C. Courcoubetis, and D. Dill, “Model-checking in dense real-time,” *Information and Computation*, vol. 104, no. 1, pp. 2–34, 1993. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0890540183710242>
- [30] P. Bouyer, “An introduction to timed automata,” in *Formal Modeling and Analysis of Timed Systems*, ser. Lecture Notes in Computer Science. Springer, 2006, vol. 3829, pp. 1–26.
- [31] S. Gil, E. Kamburjan, P. Talasila, and P. G. Larsen, “An architecture for coupled digital twins with semantic lifting,” *Software and Systems Modeling*, vol. 24, no. 5, pp. 1379–1404, 2025. [Online]. Available: <https://doi.org/10.1007/s10270-024-01221-w>
- [32] E. Karabulut, S. F. Pileggi, P. Groth, and V. Degeler, “Ontologies in digital twins: A systematic literature review,” *Future Generation Computer Systems*, vol. 153, pp. 442–456, 2024. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0167739X23004739>
- [33] C. Steinmetz, G. N. Schroeder, A. Sulak, K. Tuna, A. Binotto, A. Retberg, and C. E. Pereira, “A methodology for creating semantic digital twin models supported by knowledge graphs,” in *2022 IEEE 27th International Conference on Emerging Technologies and Factory Automation (ETFA)*, IEEE. Stuttgart, Germany: IEEE, 2022, pp. 1–7.
- [34] N. F. Noy and D. L. McGuinness, “Ontology development 101: A guide to creating your first ontology,” 2001, technical Report KSL-01-05.
- [35] R. Stevens, C. Goble, and S. Bechhofer, “Ontology-based knowledge representation for bioinformatics,” *Briefings in Bioinformatics*, vol. 1, no. 4, pp. 398–414, 2000.
- [36] A. Hogan, E. Blomqvist, M. Cochez, C. d’Amato, G. d. Melo, C. Gutierrez, S. Kirrane, J. E. L. Gayo, R. Navigli, S. Neumaier *et al.*, “Knowledge graphs,” *ACM Computing Surveys*, vol. 54, no. 4, pp. 1–37, 2021.
- [37] V. K. Kommineni, B. König-Ries, and S. Samuel, “From human experts to machines: An LLM supported approach to ontology and knowledge graph construction,” *CoRR*, vol. abs/2403.08345, 2024.
- [38] A. S. Lippolis, M. J. Saeedizade, R. Keskiä, S. Zuppiroli, M. Ceriani, A. Gangemi, E. Blomqvist, and A. G. Nuzzolese, “Ontology generation using large language models,” in *The Semantic Web – 22nd International Conference, ESWC 2025*. Springer, 2025, arXiv:2503.05388.
- [39] M. J. Saeedizade and E. Blomqvist, “Navigating ontology development with large language models,” in *The Semantic Web – 21st International Conference, ESWC 2024*, ser. Lecture Notes in Computer Science, vol. 14664. Springer, 2024, pp. 143–161.
- [40] C. Barrett and C. Tinelli, *Satisfiability Modulo Theories*. Springer, 2018.

- [41] A. Tarski, *A Decision Method for Elementary Algebra and Geometry*. University of California Press, 1951.
- [42] M. Presburger, "Über die vollständigkeit eines gewissen systems der arithmetik ganzer zahlen," in *Comptes Rendus du I Congrès des Mathématiciens des Pays Slaves*, 1929, pp. 92–101.
- [43] M. J. Fischer and M. O. Rabin, "Super-exponential complexity of presburger arithmetic," *SIAM-AMS Proceedings*, vol. 7, pp. 27–41, 1974.
- [44] R. Piskac, L. de Moura, and N. Björner, "Deciding effectively propositional logic using DPLL and substitution sets," *Journal of Automated Reasoning*, vol. 44, no. 4, pp. 401–424, 2010.
- [45] L. de Moura and N. Björner, "Z3: An efficient smt solver," in *Tools and Algorithms for the Construction and Analysis of Systems*, ser. Lecture Notes in Computer Science, vol. 4963. Springer, 2008, pp. 337–340.
- [46] D. Giannakopoulou, T. Pressburger, A. Mavridou, and J. Schumann, "Automated formalization of structured natural language requirements," *Information and Software Technology*, vol. 137, p. 106590, 2021.
- [47] Dassault Systèmes, "Catia stimulus," <https://www.3ds.com/products/catia/stimulus>, Dassault Systèmes, 2026, accessed: 2026-04-07. [Online]. Available: <https://www.3ds.com/products/catia/stimulus>
- [48] RTCA SC-205 and EUROCAE WG-12, "Software considerations in airborne systems and equipment certification," RTCA, Inc., Washington, DC, USA, Tech. Rep. DO-178C / ED-12C, December 2011.
- [49] IEC Technical Committee 65, "Functional safety — safety instrumented systems for the process industry sector," International Electrotechnical Commission, Geneva, Switzerland, Tech. Rep. IEC 61511-1:2016, 2016.
- [50] R. G. Allen, L. S. Pereira, D. Raes, and M. Smith, "Crop evapotranspiration — guidelines for computing crop water requirements," Food and Agriculture Organization of the United Nations, Rome, Italy, Tech. Rep. FAO Irrigation and Drainage Paper No. 56, Rev. 1, 2025.
- [51] I. Grangel-González, L. Halilaj, G. Coskun, S. Auer, D. Collarana, and M. Hoffmeister, "Towards a semantic administrative shell for industry 4.0 components," in *Proceedings of the IEEE 10th International Conference on Semantic Computing (ICSC)*. IEEE, 2016, pp. 230–237.
- [52] J. Arm, T. Beneš, P. Marcon, Z. Bradáč, and T. Schröder, "Automated design and integration of asset administration shells in components of industry 4.0," *Sensors*, vol. 21, no. 6, p. 2004, 2021.
- [53] I. Grangel-González, L. Halilaj, and S. Lohmann, "Semantic interoperability of digital twins: Ontology-based capability checking in aas modeling framework," in *Proceedings of the IEEE International Conference on Industrial Informatics*. IEEE, 2023.
- [54] A. Ayoub, A. Wahba, A. Salem, and M. Sheirah, "Code synthesis for timed automata: A comparison using case study," in *Abstract State Machines, Alloy, B and Z*, M. Frappier, U. Glässer, S. Khurshid, R. Laleau, and S. Reeves, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2010, pp. 403–403.
- [55] J. J. Dekker, "Generating a google go framework from an uppaal model," Bachelor's Thesis, Radboud University Nijmegen, 2014, accessed: 2025-09-08. [Online]. Available: https://www.cs.ru.nl/bachelors-theses/2014/Jip_Dekker_0403546_Generating_a_Google_Go_framework_from_an_Uppaal_model.pdf
- [56] A. Leizgold, "Uppaal2c: Generate c code from an uppaal model," 2021, gitHub repository. Accessed: 2025-09-08. [Online]. Available: <https://github.com/arieleizi/UPPAAL2C>
- [57] S. Cho, I. Kang, and J. H. Kim, "From timed automata to go: Formally verified code generation and runtime monitoring for cyber-physical systems," *IEEE Access*, vol. 13, pp. 161 729–161 749, 2025, accessed: 2025-09-08. [Online]. Available: <https://doi.org/10.1109/ACCESS.2025.3608215>
- [58] S. Feo-Arenis, M. Vujinović, and B. Westphal, "On implementable timed automata," in *Formal Techniques for Distributed Objects, Components, and Systems*, A. Gotsman and A. Sokolova, Eds. Cham: Springer International Publishing, 2020, pp. 78–95.
- [59] M. G. Seok, W. J. Tan, W. Cai, and D. Park, "Digital-twin consistency checking based on observed timed events with unobservable transitions in smart manufacturing," *IEEE Transactions on Industrial Informatics*, vol. 19, no. 4, pp. 6208–6219, 2023.
- [60] S. Hallé, C. Soueidi, and Y. Falcone, "Leveraging runtime verification for the monitoring of digital twins," in *Proceedings of the Workshop on Applications of Formal Methods and Digital Twins (FMDT'23), co-located with FM 2023*, ser. CEUR Workshop Proceedings, vol. 3507, 2023. [Online]. Available: <https://ceur-ws.org/Vol-3507/paper1.pdf>
- [61] P. Muñoz, J. Troya, and A. Vallecillo, "Towards measuring digital twins fidelity at runtime," in *Proceedings of the ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems*, ser. MODELS Companion '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 507–512. [Online]. Available: <https://doi.org/10.1145/3652620.3688267>
- [62] E. Kamburjan, C. C. Din, R. Schlatte, S. L. T. Tarifa, and E. B. Johnsen, "Twinning-by-construction: Ensuring correctness for self-adaptive digital twins," in *Leveraging Applications of Formal Methods, Verification and Validation. Verification Principles: 11th International Symposium, ISO LA 2022, Rhodes, Greece, October 22–30, 2022, Proceedings, Part I*. Berlin, Heidelberg: Springer-Verlag, 2022, p. 188–204. [Online]. Available: https://doi.org/10.1007/978-3-031-19849-6_12
- [63] R. Cleaveland, "On automatically explaining bisimulation inequivalence," in *Computer-Aided Verification, 2nd International Workshop, CAV 1990*, ser. Lecture Notes in Computer Science, vol. 531. Springer, 1991, pp. 364–372.
- [64] X. Ma, Q. Qi, and F. Tao, "An ontology-based data-model coupling approach for digital twin," *Robot. Comput.-Integr. Manuf.*, vol. 86, no. C, Apr. 2024. [Online]. Available: <https://doi.org/10.1016/j.rcim.2023.102649>
- [65] C. Boje, A. Guerriero, S. Kubicki, and Y. Rezgui, "Towards a semantic construction digital twin: Directions for future research," *Automation in Construction*, vol. 114, p. 103179, 2020.
- [66] M. Elhajj, "Semantic foundations for digital twins: the contribution of ontological analysis," *Frontiers in Computer Science*, vol. Volume 8 - 2026, 2026. [Online]. Available: <https://www.frontiersin.org/journals/computer-science/articles/10.3389/fcomp.2026.1757450>
- [67] B. Oakes, C. Gomes, E. Kamburjan, G. Abbiati, E. Ecem Bas, and S. Engelsgaard, "Towards ontological service-driven engineering of digital twins," in *Proceedings of the ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems*, ser. MODELS Companion '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 464–469. [Online]. Available: <https://doi.org/10.1145/3652620.3688261>
- [68] F. Farias et al., "Dtag: A methodology for aggregating digital twins using the wotdt ontology," *Applied Sciences*, vol. 14, no. 13, p. 5960, 2024. [Online]. Available: <https://doi.org/10.3390/app14135960>
- [69] S. R. Bader and M. Maleshkova, "The semantic asset administration shell," *arXiv preprint arXiv:1909.00690*, 2019.
- [70] N. Zhang, R. Bahsoon, N. Tziritas, and G. Theodoropoulos, "Knowledge equivalence in digital twins of intelligent systems," *ACM Trans. Model. Comput. Simul.*, vol. 34, no. 1, Jan. 2024. [Online]. Available: <https://doi.org/10.1145/3635306>
- [71] L. Huang, L. R. Varshney, and K. E. Willcox, "Formal verification of digital twins with tla and information leakage control," 2024. [Online]. Available: <https://arxiv.org/abs/2411.18798>
- [72] A. Rouatbi, E. Syriani, and B. Oakes, "DTChecker: A real-time signal monitoring and property specification tool for digital twins," in *2025 ACM/IEEE 28th International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C)*, 2025, pp. 84–88.
- [73] C. Gomes, C. Thule, D. Broman, P. G. Larsen, and H. Vangheluwe, "Co-simulation: A survey," *ACM Computing Surveys*, vol. 51, no. 3, pp. 1–33, 2018.
- [74] J. Fitzgerald, P. G. Larsen, and K. Pierce, "Multi-modelling and co-simulation in the engineering of cyber-physical systems: Towards the digital twin," in *From Software Engineering to Formal Methods and Tools, and Back*. Springer, 2019, pp. 40–55.
- [75] L. Brodo, G. Scalora, and S. Henkler, "Towards a digital twin framework for secure and efficient cyber-physical transportation systems," in *Proceedings of the 1st Workshop on Modeling and Verification for Secure and Performant Cyber-Physical Systems*, ser. MoVe4SPS '25. New York, NY, USA: Association for Computing Machinery, 2025. [Online]. Available: <https://doi.org/10.1145/3735948.3736156>
- [76] J. Woodcock, C. Gomes, H. D. Macedo, and P. G. Larsen, "Uncertainty quantification and runtime monitoring using environment-aware digital twins," in *Leveraging Applications of Formal Methods, Verification and Validation (ISO LA 2020), Part IV*, ser. LNCS, vol. 12479. Springer, 2021, pp. 72–87.